

Course

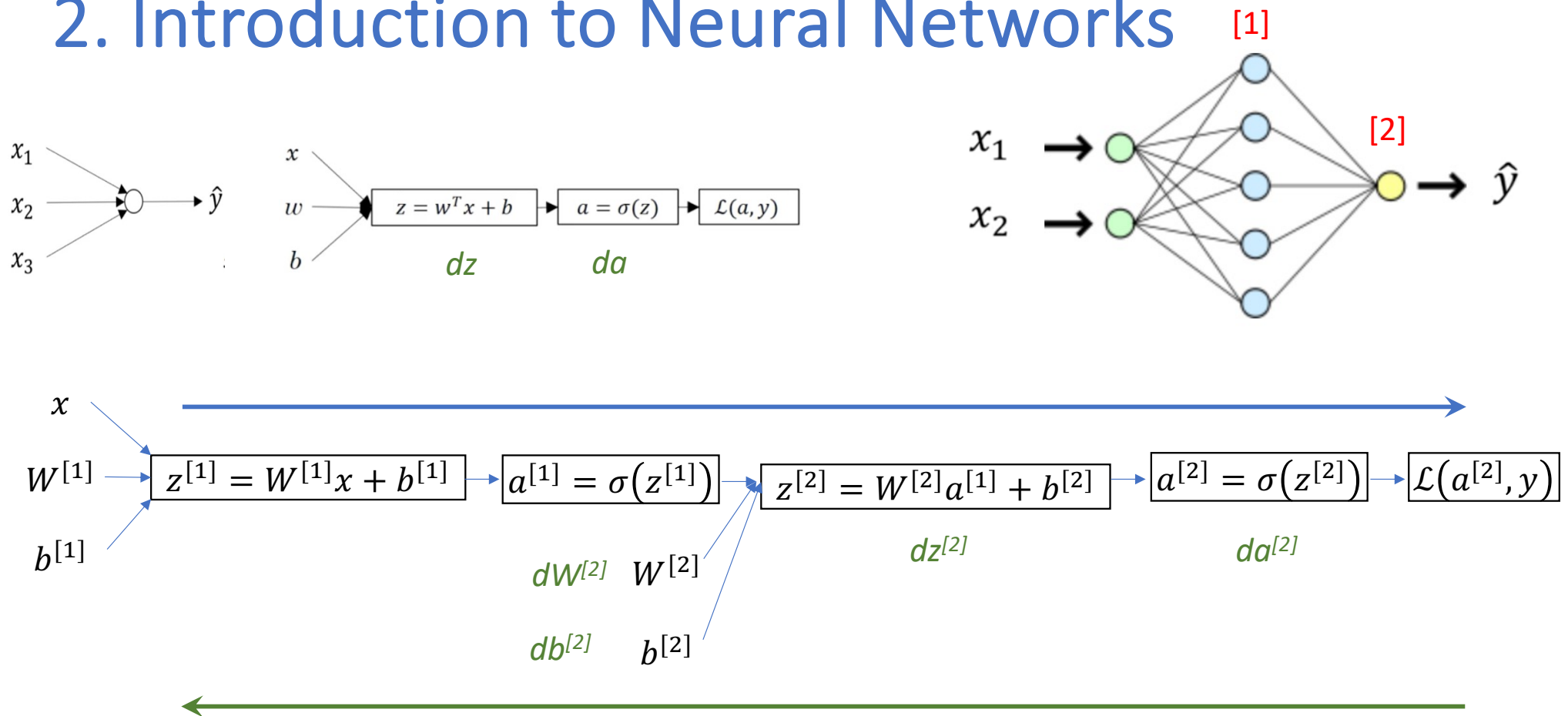
« *Deep Learning* »

Sid-Ahmed Berrani

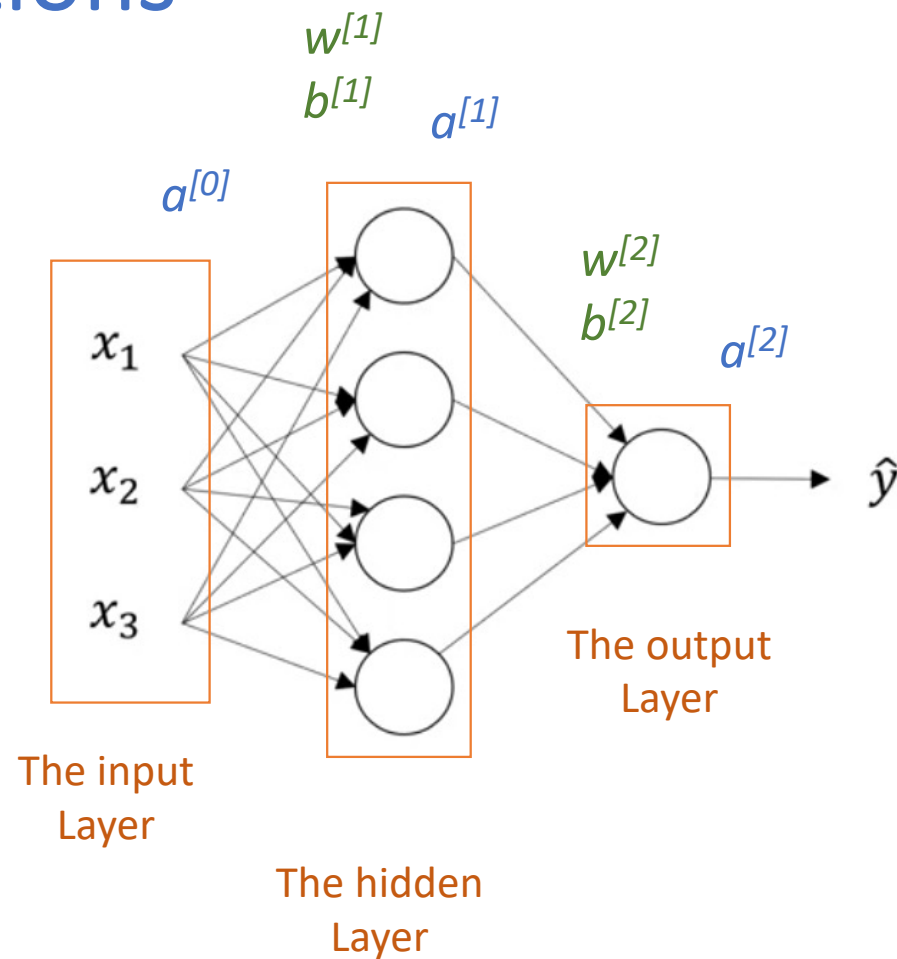
ENSIA

2024 – 2025

2. Introduction to Neural Networks

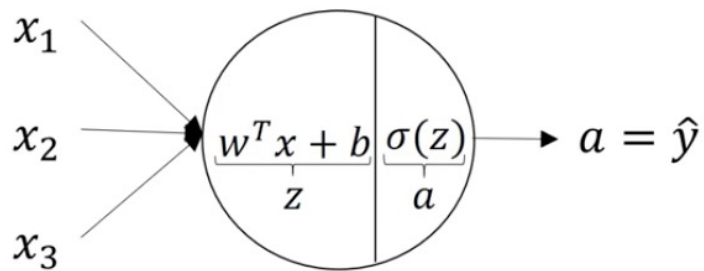


2. Introduction to Neural Networks: Notations



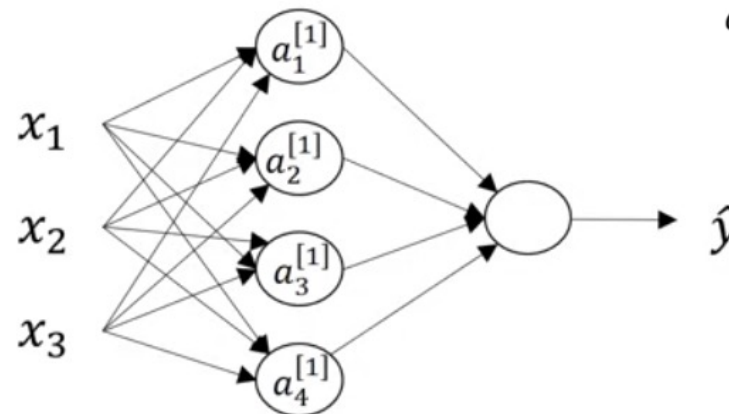
A two-layer neural network

2. Introduction to Neural Networks: Notations



$$z = w^T x + b$$

$$a = \sigma(z)$$

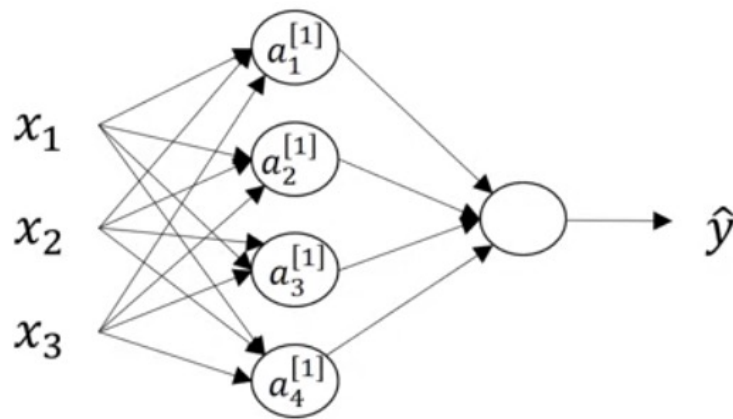


$$z_1^{[1]} = w_1^{[1]T} x + b_1^{[1]},$$

$$a_1^{[1]} = \sigma(z_1^{[1]})$$

2. Introduction to Neural Networks

Computing outputs



$$z_1^{[1]} = w_1^{[1]T} x + b_1^{[1]}, \quad a_1^{[1]} = \sigma(z_1^{[1]})$$

$$z_2^{[1]} = w_2^{[1]T} x + b_2^{[1]}, \quad a_2^{[1]} = \sigma(z_2^{[1]})$$

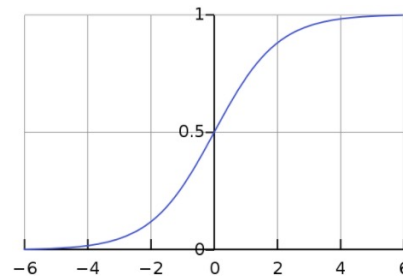
$$z_3^{[1]} = w_3^{[1]T} x + b_3^{[1]}, \quad a_3^{[1]} = \sigma(z_3^{[1]})$$

$$z_4^{[1]} = w_4^{[1]T} x + b_4^{[1]}, \quad a_4^{[1]} = \sigma(z_4^{[1]})$$

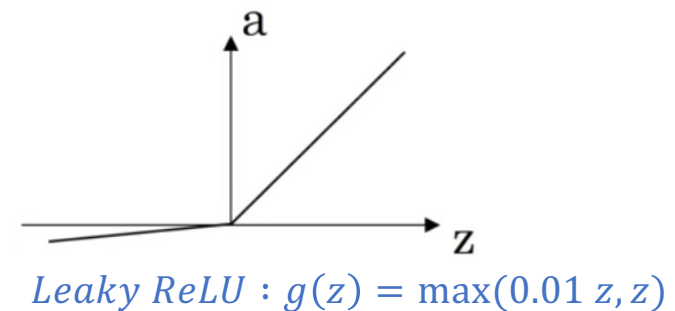
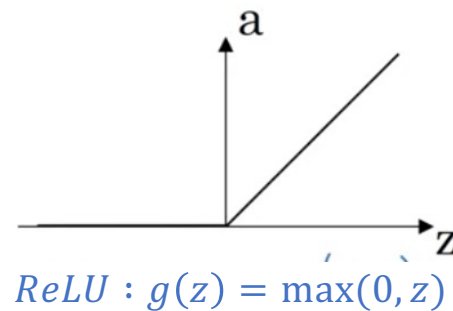
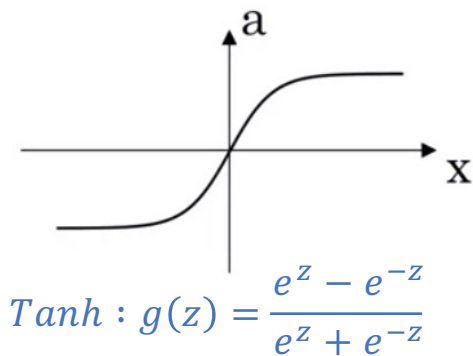
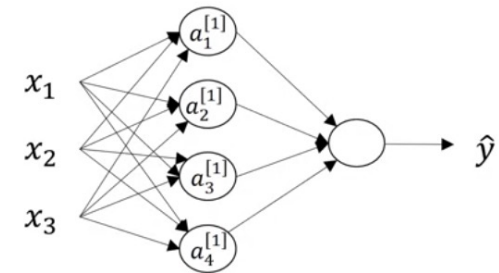
2. Introduction to Neural Networks: Activation Functions

- The activation function used so far: Sigmoid function

$$\begin{aligned} z^{[1]} &= W^{[1]}x + b^{[1]} \\ a^{[1]} &= \cancel{\sigma(z^{[1]})} g(z^{[1]}) \\ z^{[2]} &= W^{[2]}a^{[1]} + b^{[2]} \\ a^{[2]} &= \cancel{\sigma(z^{[2]})} g(z^{[2]}) \end{aligned}$$



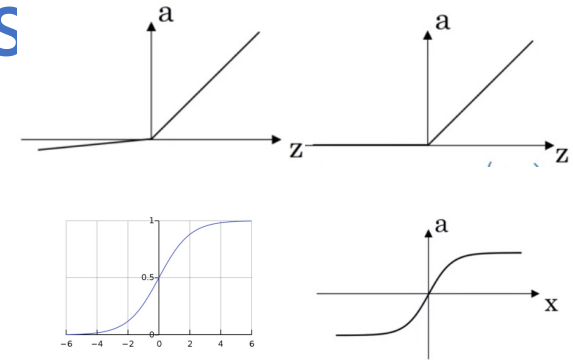
$$\text{Sigmoid} : g(z) = \frac{1}{1 + e^{-z}}$$



2. Introduction to Neural Networks

Activation Functions

- These are the most widely used activation functions.
- They achieve the best performance.
- All these activation functions are non-linear.
=> Why do we need the non-linearity?



Remember:

- Often a linear function cannot explain complex relationship between variables.
- Non-linear means the graph is not a straight line. The graph of a non-linear function is a curved line. A curved line is a line whose direction constantly changes.
- The slope of a linear function is the same no matter where on the line it is measured.
- There is no single slope for a non-linear function

2. Introduction to Neural Networks: Activation Functions

- **Why do we need non-linear activation functions?**

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

$$a^{[1]} = \cancel{\sigma(z^{[1]})} \quad z^{[1]}$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[2]} = \cancel{\sigma(z^{[2]})} \quad z^{[2]}$$

$z^{[2]}$

The composition of two linear functions is itself a linear function.

- Here the linear activation function: $g(z) = z$.

$$a^{[1]} = z^{[1]} = w^{[1]}x + b^{[1]}$$

$$a^{[2]} = z^{[2]} = w^{[2]}a^{[1]} + b^{[2]} = w^{[2]}(w^{[1]}x + b^{[1]}) + b^{[2]} = w^{[2]} w^{[1]}x + w^{[2]} b^{[1]} + b^{[2]}$$

$$= w'x + b'$$

2. Introduction to Neural Networks: Parameters and hyper-parameters

- Learning allows to tune the **parameters**: $W^{[1]}$, $b^{[1]}$, $W^{[2]}$, $b^{[2]}$, $W^{[3]}$, $b^{[3]}$...
- **Hyper-parameters** :
 - Learning rate
 - Number of hidden layers
 - Number of hidden neurons
 - Number of iterations
 - The choice of activation function
 - ... and many other hyper-parameters we will see later...

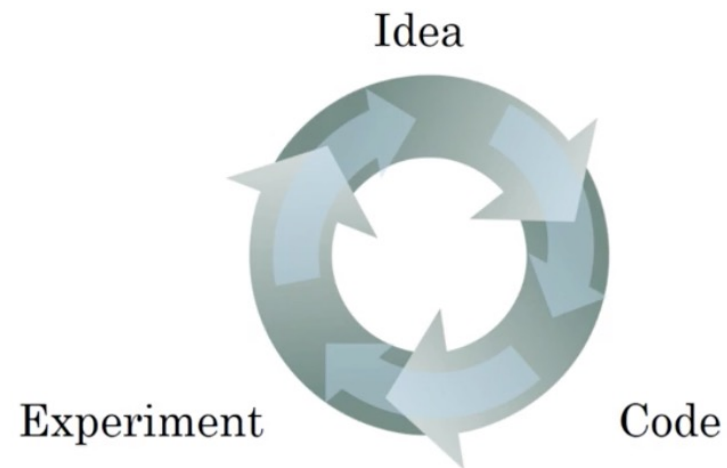
2. Introduction to Neural Networks:

Building a NN: An iterative process

Building a NN requires to define and fine-tune the hyperparameters in order to learn the parameters.

In practice :

- It is an iterative process!



2. Introduction to Neural Networks: Train/Dev/Test sets

- Available data are most often split into 3 parts:
 1. Training set
 2. Development set (cross validation set)
 3. Test set.



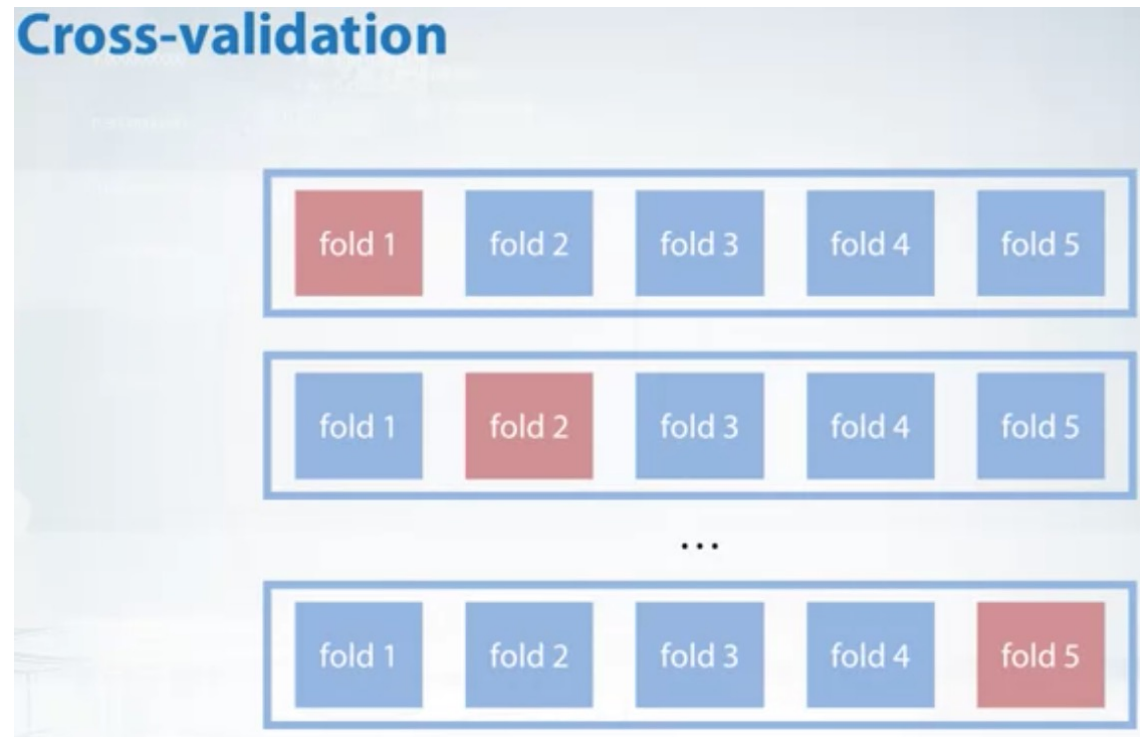
Train/Test: 70/30% 80/20%

Train/Dev/Test: 60/20/20% 80/10/10% 80/5/15%

If we have 1,000,000 examples: 1% for dev, 1% for test are sufficient.

2. Introduction to Neural Networks: Train/Dev/Test sets

- When the data set is small, we can use the cross-validation procedure:



2. Introduction to Neural Networks: Train/Dev/Test sets

- It is important to make sure that the dev/test data come from the same distribution as the train data.



2. Introduction to Neural Networks: Normalization of input vectors

- Two steps:
 - Mean = (0,0)
 - Variance normalization.
- Normalization parameters used for test data must be those computed for train data.

